VoiceOps 2026 — ZentroMart Voice Support Agent

Oct 31, 2026 · Dharmana Purvi

Summary

I built ZentroMart's support agent as one pipeline: Gnani STT → an LLM agent with RAG and tools → Gnani TTS. Grounding the LLM in the knowledge base took practice-question accuracy from **12/52 (23.1%) to 52/52 (100%)**. The tool-using conversation agent then got **96/96 turns** and **23/23 conversations** fully correct.

Step	Evaluation set	Fully correct	Decision accuracy	Fact recall
1. Prompt-only LLM	questions.csv (52)	12 / 52 (23.1%)	25.0%	30.8%
2. RAG over kb/	questions.csv (52)	52 / 52 (100%)	100%	100%
3. Agent + tools + memory	conversations.csv (96 turns)	96 / 96 (100%)	100%	100%
4. Voice (Gnani STT/TTS)	Live calls, own voice	End to end in English and Hindi	—	—

Stack

Component	Choice
LLM	`nvidia/nemotron-3.5-super-vl-120b-a12b` via the NVIDIA API (OpenAI-compatible client)
Embeddings	`intfloat/multilingual-e5-base` (handles English, Hindi and Hinglish)
Retrieval	Hybrid dense + BM25, fused with reciprocal rank fusion; numpy index cached to disk
Tools	`lookup_order`, `create_ticket` (provided) + `search_kb`, `date_calc`, `final_answer` (mine)
Speech	Gnani `prisma-v3.0` STT (auto language detection), Gnani `timbre-v2.5` TTS, voice Kaveri

Component	Choice
Interfaces	One `run.py`: Gradio web-app call (default) or batch over an audio folder; optional terminal calls in `src/`

Step 1 — Answer a question with an LLM

Without documents, the LLM answered **12 of 52** practice questions correctly. It almost never guessed a policy; instead it said "I don't have that information" and escalated. That kept violations at 0 but failed 40 of 46 answerable questions.

Tech used: Python · `openai` SDK (OpenAI-compatible client) → NVIDIA API, model `nvidia/nemotron-3.5-super-vl-120b-a12b` · JSON-mode output · `python-dotenv` for keys · `pandas` · regex/Unicode scorer (`metrics.py`) · JSON answer cache · Jupyter (`step1_baseline.ipynb`)

What I built (`llm.py`)

- One system prompt makes the model ZentroMart's AI support assistant, sets today as 20 October 2026, and states its limits.
- The first reply always says the customer is talking to an AI assistant.
- The model returns JSON: `key_facts`, `decision` (answer / clarify / escalate / refuse), `calc`, `reply`, `used_sections`. Writing `key_facts` first forces exact figures into the reply.
- In Step 1 the prompt adds: "You have no access to company documents."
- Calls retry on rate limits, and every result is cached per question so reruns are cheap.

How I scored it (`metrics.py`, same scorer for Steps 1 and 2)

- *Decision accuracy*: the decision is in `acceptable_decisions`.
- *Fact recall*: share of `must_include` facts found in the reply, after normalising numbers, Devanagari digits, rupee signs and English/Hindi dates.
- *Violations*: any `must_not_include` phrase present (for example the superseded 5–7 day refund window).
- *Correct*: right decision, every fact present, no violation.

Where it failed. Of 46 questions that expect *answer*, the baseline escalated 43 and got 6 right. Its other 6 correct were the 2 clarify, 3 escalate and 1 refuse cases. It did get all 5 *unsupported* questions right, because there "I don't know" is the correct reply.

Step 2 — Ground it in the documents

With retrieval over `kb/`, the same model and prompt answered **52 of 52** questions correctly, up from 12. Every answerable question that Step 1 escalated now gets the exact figure from the right section.

Tech used: Same LLM and prompt as Step 1 · `sentence-transformers` with `intfloat/multilingual-e5-base` embeddings · `rank-bm25` (BM25Okapi) · `numpy` cosine similarity + reciprocal rank fusion · pickle-cached vector index · `pandas` for `products.csv` and `holidays.csv` · Python `datetime` date table · Jupyter (`step2_rag.ipynb`)

What I built (`rag_pipeline.py`)

- **Chunking.** Each `kb/*.md` file is split at its `[SECTION-ID]` headings (about 1,500 characters max per chunk). Each chunk keeps `doc_type`, `status`, `effective_from` and language. Each of the 70 product rows becomes one chunk (`products.csv:SKU`), so questions about a SKU find its return category and warranty.
- **Index.** Every chunk is embedded with `multilingual-e5-base` and the vectors are cached in `outputs/vector_index.pkl`. A BM25 index sits beside them.
- **Hybrid retrieval.** Dense and BM25 rankings are fused with reciprocal rank fusion (dense weighted 2×). Superseded sections get a 0.85 penalty and lower-precedence document types a small one (policy > agent guide > help > FAQ > marketing). A SKU named in the question pins its product row. Top 26 sections, at most 2 product rows.
- **Grounded prompt.** The model may answer only from the CONTEXT and must say "I don't have that information" and escalate when it is not covered. It must prefer current over superseded policy unless the order predates the new one.
- **Date table.** Python computes business days (Monday to Saturday, skipping `holidays.csv`) and calendar days from 20 October 2026 or any date in the question. The model reads end dates from this table instead of counting itself.
- **Citations.** The model lists `used_sections`. IDs not in the retrieved context are dropped, so every answer records the chunks it used.

Step 1 vs Step 2

Metric	Step 1 (no docs)	Step 2 (RAG)
Fully correct	12 / 52 (23.1%)	**52 / 52 (100%)**
Decision accuracy	25.0%	100%
Avg fact recall	30.8%	100%
Violations	0	0

Metric	Step 1 (no docs)	Step 2 (RAG)
Recall@k (gold sections retrieved)	—	0.933
Hit@k (at least one gold section)	—	1.000
MRR	—	0.641
Citation precision	—	0.748

The gain holds in every language: English 9 → 33 of 33, Hindi 2 → 8 of 8, Hinglish 1 → 11 of 11. The largest jumps were refund timelines (1 → 14 of 14), return windows (1 → 11 of 11) and warranty (0 → 5 of 5).

What I learned

- The baseline did not hallucinate; it refused to commit. A careful prompt alone produces a safe but useless agent. The facts have to come from the documents.
- Retrieval was not the bottleneck: hit@k was 1.0, so a gold section was always in context. Most of my iteration went into the answer stage: precedence and superseded rules, the date table, and writing `key_facts` before the reply.
- Traps in the knowledge base matter. FAQ-1 says refunds take 5–7 days for every method, but REF-2 says wallet refunds are instant. The precedence rule and the `must_not_include` check caught these.
- Dates are where an LLM fails quietly. Moving business-day counting into Python removed a whole class of off-by-one errors.

Step 3 — Hold a conversation

The tool-using agent handled all **96 turns** across the **23 practice conversations** correctly. It made all 54 expected tool calls and created no unexpected tickets.

Tech used: Same LLM via OpenAI-style function calling (tool use) · provided `tools.py` (`lookup_order` , `create_ticket`) over the `data/*.csv` tables · Step 2 hybrid retrieval as the `search_kb` tool · Python `datetime` for `date_calc` (Mon–Sat business days, holidays skipped) · in-memory conversation history · `pandas` scorer for decisions, facts and tool calls

What I built (`agent.py`)

- **Agent loop.** For each customer message the LLM calls tools until it calls `final_answer` with the decision, reply and sections used.

- **Tools.** `search_kb` (Step 2 retrieval, 16 sections per query in English), `lookup_order` (provided, with identity check), `create_ticket` (provided, for handovers), `date_calc` (business or calendar days in Python) and `final_answer`.
- **Memory.** The full turn history, verified orders and created tickets persist across turns, so "how long will the refund take?" resolves to the order already discussed. Bulky search results are kept only for the current turn to save context.
- **Standing rules.** Agent guides (style, identity verification, escalation and handoff) are always in the system prompt, so a handover rule applies even when no search returns it.
- **Safety enforced in code, not left to the prompt.**
  - The order is shown only after the last 4 phone digits or the delivery pincode match, with at most 2 attempts.
  - The pincode is never shown to the LLM.
  - One ticket per issue; a handover is confirmed only after `created=True`.
  - Dates go in as `{date1}` placeholders filled by Python.
  - The agent introduces itself as an AI on turn 1.
- **No canned answers.** When a tool fails or returns nothing, the agent passes on the tool's actual reason instead of a hardcoded message.

Results

Metric	Value
Turns correct (reply + tools)	96 / 96 (100%)
Reply correct, ignoring tools	100%
Decision accuracy	100%
Avg fact recall	100%
Violations	0
Expected tool calls made	54 / 54
Unexpected tickets	0
Conversations fully correct	23 / 23
Recall@k / Hit@k	0.970 / 0.979
MRR / Citation precision	0.258 / 0.038

Breakdown	Turns, all 100% correct
By decision	answer 36 · clarify 48 · escalate 6 · refuse 6
By language	English 33 · Hindi 21 · Hinglish 42
By scenario	cancel shipped 8 · cancel unshipped 8 · compensation demand 8 · express late credit 8 · failed verification 8 · high-value damage 4 · human requested 8 · refund status 8 · repeat contact 8 · return eligibility 10 · topic switch 10 · where is my order 8

Example: high-value damage, Hinglish (conversation turns 1–4)

1. *Customer:* "Hello, mujhe order ke baare mein help chahiye." → **clarify**: introduces itself as ZentroMart's AI assistant and asks for the 8-digit order ID.
2. *Customer:* "Order number hai 8978 2837." → **clarify**: `lookup_order(89782837)` without verification, then asks for the last 4 phone digits or the pincode.
3. *Customer:* "Last four digits 4533 hain." → **answer**: verified; the order was delivered on 15 October 2026.
4. *Customer:* "Replacement approve kar do na?" → **escalate**: the TV costs ₹31,649, and damage claims above ₹10,000 go to the Priority Resolution team. `create_ticket` returned a ticket ID, and only then did the agent say a human would call back.

Step 4 — Give it a voice

The Step 3 agent now takes a phone-style call: Gnani STT turns the caller's audio into text, the unchanged agent answers, and Gnani TTS speaks the reply. Only Gnani is used for speech; there is no other STT or TTS, even as a backup. `GNANI_API_KEY` is read from `.env` and is never in the code.

Tech used: Gnani `gnani-vachana` Python SDK: STT `prisma-v3.0` (`indic-auto` , fallback `en-IN`) and TTS `timbre-v2.5` , voice Kaveri · the Step 3 agent unchanged · `sounddevice` + `soundfile` + `numpy` for mic capture, voice activity detection and WAV handling · Python `wave` for splitting and joining audio · `ThreadPoolExecutor` for parallel TTS · `gradio` web app · macOS `afplay` for playback

What I built (`voice.py`)

- **Ears (Gnani STT).** `prisma-v3.0` with `indic-auto` language detection (candidates `hi-IN` , `en-IN`). A topic hint, keywords (ZentroMart, refund, pincode, Assured…) and the previous transcript as context improve recognition of domain words and short replies

such as "4265". Clips longer than 28 s are split and transcribed in pieces. If `prisma-v3.0` is unavailable, it falls back to Gnani's default model with `en-IN`.

- **Brain.** `VoiceAgent` wraps the same Step 3 `Agent`, one instance per call, so memory lasts the whole call.
- **Voice (Gnani TTS).** `timbre-v2.5`, voice Kaveri, 24 kHz WAV. The TTS language follows the reply's script: `hi-IN` for Devanagari, `en-IN` for English, auto for Hinglish. Long replies are synthesised sentence by sentence in parallel and joined.
- **Written for the ear.** Order IDs are read in groups of four, rupee amounts are spoken as words, ticket IDs are shortened, and dates include the year.
- **Unclear audio.** An empty transcript or only fillers ("um", "hmm") gets "Sorry, I did not catch that…" in the caller's last language, without calling the LLM. One real word ("haan", "yes") counts as an answer.
- **Live-call mode.** Replies are kept to 2–3 short sentences. Greetings and thanks are answered instantly. The knowledge base is searched before the first LLM call, and there is at most one ticket per call. If the LLM fails, the caller hears an apology and a handover, not silence.

Run command (one file, run.py)

```
python run.py                                    # web app: call starts, ask
by mic
python run.py --audio_dir <folder_of_audio>     # organiser batch run ->
sample_outputs/run
python run.py --audio_dir <folder> --conversation   # files in name order =
one call
```

For each audio file (.wav, .mp3, .m4a, .flac, .ogg, .aac) it writes `<name>.reply.wav`, plus `<name>.json` with the transcript, reply text, decision, tool calls and the full text of the document chunks used. It also writes `results.csv` and `results.jsonl` for the folder.

Live call, 31 October 2026 (my own voice, `sample_outputs/live_call_2026-10-31/`)

Turn	Caller said (STT)	Agent	Turn time
1	"Hi, I want a help with my order." (en-IN)	clarify: AI intro, asks for order ID	6.9 s
2	"My order ID is 18599253."	clarify: `lookup_order`, asks to verify	9.9 s
3	"4265."	answer: verified, delivered 12 October 2026	9.5 s

Turn	Caller said (STT)	Agent	Turn time
4	"क्या मैं … रनिंग सीकर्स रिटर्न कर सकती हूँ?" (hi-IN)	answer in Hindi: 15-day fashion window, ends 27 October 2026; cites RET-2, RET-4, PLS-2	34.4 s
5	"और रिफंड कितने दिन में आएगी?"	answer: follow-up on the same order; wallet refund within 2 hours (REF-2)	20.8 s
6	"Okay, thank you, bye."	closes the call	82.7 s

The call switched from English to Hindi mid-call. STT returned "सीकर्स" for "Sneakers", but the agent still matched the product from the verified order. STT took 0.6–3.7 s per turn; most of the turn time was the LLM and its tool calls.

What went wrong and what I would fix

Answers are right on the practice set. The weak spots are retrieval bookkeeping in Step 3, voice latency, and how far the practice scores carry over to unseen audio.

Problem	Evidence	Fix
Step 3 retrieval metrics look poor	MRR 0.258 and citation precision 0.038, even with recall@k 0.970 and every turn correct. Standing rules (KB-00, STY, ESC, IDV) are logged first in a fixed order, so a gold section like IDV-2 lands at rank ~15. The agent follows these guides but rarely cites them, and some citations carry a file prefix (`pol_returns_v4.md:RET-2`) that does not match the gold ID.	Log `search_kb` hits separately from standing rules and rank only those. Strip file prefixes from cited IDs. Ask the agent to cite the guide sections it applied.
Slow voice turns	Policy turns took 20–34 s, and the closing "thank you, bye" took 82.7 s. The batch run and push-to-talk call build `VoiceAgent()` without fast mode, so they skip instant small talk and search 16 sections per query (the web app now uses fast mode).	Use fast mode in every voice entry point. Speak a short filler while the agent thinks, stream TTS sentence by sentence, and catch goodbye before the LLM.

Problem	Evidence	Fix
Replies too long for the ear	The Hindi return answer in turn 4 ran past 300 characters. The refund turn gave both "within 2 business days of pickup" and "at pickup for Plus" before the actual answer.	Apply the 2–3-sentence voice rule in every voice path, and lead with the one figure the caller asked for.
Steps 1–2 scored on clean text	Scoring used the `question` column, but the organisers test with audio.	Rerun Steps 1–2 on `asr_text` and report the drop, so ASR robustness is measured, not assumed.
Risk of tuning to the practice set	The prompt grew rules for specific cases (warranty months, "by when" dates) while iterating on 52 questions.	Hold out part of the practice set, add paraphrased and noisy versions, and check every new rule against the held-out cases.

Bonus work and how to run

Bonus work

- **Web app** (`web_app.py, Gradio, started by run.py`): record with the browser mic, hear the reply, see the chat. Say "bye" to end; "New call" starts fresh. Every turn's audio and JSON is saved.
- **Hands-free phone mode** (`src/call_handsfree.py`): voice activity detection with noise calibration starts recording at speech and stops after about 0.7 s of silence, with no button presses. It uses fast mode, cached greeting audio and streamed TTS.
- **Hindi and Hinglish end to end**: STT auto-detects the language, the agent answers in the caller's language, and TTS picks the matching voice language.
- **Provider-agnostic LLM**: one `.env` switch between NVIDIA, Groq, Cerebras, Gemini, OpenRouter or a local Ollama model, with rate-limit pacing.
- **Reproducible evaluation**: every LLM answer is cached and scored offline, and `data_validation.py` checks row counts, keys and KB section IDs before building.

How to run

```
pip install -r requirements.txt
cp .env.example .env                          # fill GNANI_API_KEY and LLM_*
values
python run.py                                 # web app: call starts, ask by mic
python run.py --audio_dir <folder>            # organiser batch run ->
sample_outputs/run
```

Steps 1–2 run in `src/step1_baseline.ipynb` and `src/step2_rag.ipynb`, and Step 3 in `src/step3_conversations.py`. Results are saved to `sample_outputs/evaluation/step1_vs_step2.csv` and `sample_outputs/evaluation/step3_metrics_v4.csv`.